INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)
Dundigal - 500 043, Hyderabad, Telangana

DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DATABASE MANAGEMENT SYSTEMS LABORATORY

Name of the Staff :Dr.B.Venkateswara Rao **Emp Id:11207**

EXERCISE –2: QUERIES USING DDL AND DML **Date:03/08/2026**

Aim:
To study and execute **DDL (Data Definition Language)** and **DML (Data Manipulation Language)** commands in Oracle SQL for creating, modifying, and managing database tables and records.

DDL and DML Commands

Purpose:
To study and execute **Data Definition Language (DDL)** and **Data Manipulation Language (DML)** commands in SQL for creating, modifying, and managing table records in a database.

DDL Commands – Purpose:

Command	Purpose
CREATE	To create a new table or database object.
ALTER	To modify the structure of an existing table.
DROP	To permanently remove a table or database object.
TRUNCATE	To remove all records from a table while retaining its structure.
RENAME	To change the name of an existing table.

DML Commands – Purpose

Command	Purpose
INSERT	To add new student records to a table.
UPDATE	To modify existing student records.
DELETE	To remove existing student records from a table.

DDL and DML commands in SQL — syntax .

DDL Commands

1. CREATE TABLE

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
   column3 datatype
);
```

2. ALTER TABLE – Add Column

```
ALTER TABLE table_name
ADD column_name datatype;
```

3. ALTER TABLE – Modify Column

```
ALTER TABLE table_name
MODIFY column_name datatype;
```

4. ALTER TABLE – Drop Column

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

5. DROP TABLE

```
DROP TABLE table_name;
```

6. TRUNCATE TABLE

```
TRUNCATE TABLE table_name;
```

7. RENAME TABLE

```
RENAME old_table_name TO new_table_name;
```

DML Commands

1. INSERT

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

2. UPDATE

```
UPDATE table_name
SET column1 = value1
WHERE condition;
```

3. DELETE

```
DELETE FROM table_name
WHERE condition;
```

4. SELECT

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Transaction Commands

COMMIT

```
COMMIT;
```

ROLLBACK

```
ROLLBACK;
```

SAVEPOINT

```
SAVEPOINT savepoint_name;
```

Common User Creation

Execute these commands as SYSTEM:

```
CREATE USER USER1 IDENTIFIED BY user123;

GRANT CREATE SESSION, RESOURCE TO USER1;

GRANT ALL ON EMPLOYEE TO USER1;
```

Note: GRANT ALL ON EMPLOYEE TO USER1;
must be executed by the owner of the EMPLOYEE table.

1.GRANT CREATE SESSION, RESOURCE TO USER1;

This gives **USER1** permission to:

- CREATE SESSION → **log in/connect to the Oracle database**
- RESOURCE → gives privileges needed to **create database objects** such as tables, views, procedures, etc. (via the RESOURCE role in Oracle 11g)

USER1 can log in and create database objects.

2. GRANT ALL ON EMPLOYEE TO USER1;

This gives **USER1** permissions on the EMPLOYEE table.

ALL includes privileges such as:

- SELECT → view records
- INSERT → insert records
- UPDATE → modify records
- DELETE → delete records

---

1. EMPLOYEE TABLE

a. Create a user and grant all permissions

```
CREATE USER USER1 IDENTIFIED BY user123;
GRANT CREATE SESSION, RESOURCE TO USER1;
GRANT ALL ON EMPLOYEE TO USER1;
```

Here shift to user1:

```
SQL> connect;
Enter user-name: user1
Enter password:
Connected.
SQL>
```

b. Insert any three records and use ROLLBACK

```
INSERT INTO SYSTEM.EMPLOYEE VALUES (101, 'Ravi', 'Manager', 50000,50,5000);
INSERT INTO SYSTEM.EMPLOYEE VALUES (102, 'Kiran', 'Clerk', 25000,10,3000);
INSERT INTO SYSTEM.EMPLOYEE VALUES (103, 'Suresh', 'Analyst', 35000,50,4000);

SELECT * FROM System.EMPLOYEE;
```

{write expected output here}

```
ROLLBACK;

SELECT * FROM SYSTEM.EMPLOYEE;
```

{write expected output here}

Observation: The three inserted records are removed after ROLLBACK.

c. Add PRIMARY KEY and NOT NULL constraints

```
ALTER TABLE EMPLOYEE
ADD CONSTRAINT emp_pk PRIMARY KEY (EMPNO);
```

For NOT NULL:

```
ALTER TABLE EMPLOYEE
MODIFY ENAME VARCHAR2(20) NOT NULL;
```

d. Insert NULL values and verify

```
INSERT INTO EMPLOYEE
VALUES (104, NULL, 'Clerk', 20000);
```

{write expected output here}

Result: Oracle displays an error because ENAME has a NOT NULL constraint.

2. DEPARTMENT TABLE

a. Create a user and grant all permissions

```
CREATE USER USER1 IDENTIFIED BY user123;

GRANT CREATE SESSION, RESOURCE TO USER1;

GRANT ALL ON DEPARTMENT TO USER1;
```

b. Insert values and use COMMIT

```
INSERT INTO DEPARTMENT VALUES (10, 'CSE', 'HYDERABAD');
INSERT INTO DEPARTMENT VALUES (20, 'AIML', 'VIJAYAWADA');
INSERT INTO DEPARTMENT VALUES (30, 'ECE', 'GUNTUR');

COMMIT;
```

Verify:

```
SELECT * FROM DEPARTMENT;
```

{write expected output here}

c. Add UNIQUE and NOT NULL constraints

```
ALTER TABLE DEPARTMENT
ADD CONSTRAINT dept_unique UNIQUE (DEPTNAME);
ALTER TABLE DEPARTMENT
MODIFY DEPTNAME VARCHAR2(20) NOT NULL;
```

d. Insert repeated and NULL values

Repeated value:

```
INSERT INTO DEPARTMENT
VALUES (40, 'CSE', 'MACHILIPATNAM');
```

Result: Error because DEPTNAME must be unique.

NULL value:

```
INSERT INTO DEPARTMENT
VALUES (50, NULL, 'HYDERABAD');
```

Result: Error because DEPTNAME cannot be NULL.

3. DEPARTMENT TABLE

a. Create a user and grant all permissions

```
CREATE USER USER1 IDENTIFIED BY user123;

GRANT CREATE SESSION, RESOURCE TO USER1;

GRANT ALL ON DEPARTMENT TO USER1;
```

b. Insert values and use COMMIT

```
INSERT INTO DEPARTMENT VALUES (40, 'MECH', 'VIJAYAWADA');
INSERT INTO DEPARTMENT VALUES (50, 'CIVIL', 'GUNTUR');
INSERT INTO DEPARTMENT VALUES (60, 'IT', 'HYDERABAD');

COMMIT;
```

c. Delete any three records and use ROLLBACK

```
DELETE FROM DEPARTMENT
```

WHERE DEPTNO IN (40, 50, 60);

Check:

SELECT * FROM DEPARTMENT;

{write expected output here}

Now rollback:

ROLLBACK;

Verify:

SELECT * FROM DEPARTMENT;

{write expected output here}

Observation: The deleted records are restored.

d. Add PRIMARY KEY and FOREIGN KEY

Primary key:

```
ALTER TABLE DEPARTMENT
ADD CONSTRAINT dept_pk PRIMARY KEY (DEPTNO);
```

Suppose EMPLOYEE contains DEPTNO:

```
ALTER TABLE EMPLOYEE
ADD CONSTRAINT emp_dept_fk
FOREIGN KEY (DEPTNO)
REFERENCES DEPARTMENT(DEPTNO);
```

Important: The DEPTNO column must exist in EMPLOYEE,
and its datatype should match DEPARTMENT.DEPTNO.

4. SAILOR TABLE

a. Create a user and grant all permissions

CREATE USER USER1 IDENTIFIED BY user123;

GRANT CREATE SESSION, RESOURCE TO USER1;

GRANT ALL ON SAILOR TO USER1;

b. Insert records and use COMMIT

Example:

```
INSERT INTO SAILOR VALUES (1, 'RAVI', 8, 25);
INSERT INTO SAILOR VALUES (2, 'KIRAN', 7, 30);
INSERT INTO SAILOR VALUES (3, 'SURESH', 9, 28);

COMMIT;
```

Verify:

SELECT * FROM SAILOR;
{write expected output here}

c. Add SAVEPOINT after insertion
SAVEPOINT S1;

To verify the savepoint, perform another operation:

INSERT INTO SAILOR VALUES (4, 'RAHUL', 6, 22);

ROLLBACK TO S1;

Now check:

SELECT * FROM SAILOR;
{write expected output here}

Observation: Record 4 is removed, but records 1, 2 and 3 remain.

d. Add NOT NULL and PRIMARY KEY
ALTER TABLE SAILOR
ADD CONSTRAINT sailor_pk PRIMARY KEY (SID);

For NOT NULL:

ALTER TABLE SAILOR
MODIFY SNAME VARCHAR2(20) NOT NULL;